

(Almost) Tight Bounds and Existence Theorems for Confluent Flows

Jiangzhuo Chen *

Robert D. Kleinberg †

László Lovász ‡

Rajmohan Rajaraman *

Ravi Sundaram §

Adrian Vetta ¶

ABSTRACT

A flow is said to be confluent if at any node all the flow leaves along a single edge. Given a directed graph G with k sinks and non-negative demands on all the nodes of G , we consider the problem of determining a confluent flow that routes every node demand to some sink such that the maximum congestion at a sink is minimized. Confluent flows arise in a variety of application areas, most notably in networking; in fact, most flows in the Internet are confluent since Internet routing is destination based.

We present near-tight approximation algorithms, hardness results, and existence theorems for confluent flows. The main result of this paper is a polynomial-time algorithm for determining a confluent flow with congestion at most $1 + \ln(k)$ in G , if G admits a splittable flow with congestion at most 1. We complement this result in two directions. First, we present a graph G that admits a splittable flow with congestion at most 1, yet no confluent flow with congestion smaller than H_k , thus establishing tight upper and lower bounds to within an additive constant less than 1. Second, we show that it is NP-hard to approximate the con-

gestion of an optimal confluent flow to within a factor of $(\lg k)/2$, thus resolving the polynomial-time approximability to within a multiplicative constant. We also consider a demand maximization version of the problem. We show that if G admits a splittable flow of congestion at most 1, then a variant of the congestion minimization algorithm yields a confluent flow in G with congestion at most 1 that satisfies $1/3$ fraction of total demand.

We show that the gap between confluent flows and splittable flows is much smaller, if the underlying graph were k -connected. In particular, we prove that k -connected graphs with k sinks admit confluent flows of congestion less than $C + d_{\max}$, where C is the congestion of the best splittable flow, and $d_{\max}$ is the maximum demand of any node in G . The proof of this existence theorem is non-constructive and relies on topological techniques introduced in [16].

Categories and Subject Descriptors

F.2.2 [Analysis of Algorithms and Problem Complexity]: Nonnumerical Algorithms and Problems—*Routing and layout*

General Terms

Algorithms, Theory

Keywords

Approximation algorithms, confluent flow, network flow, routing, tight bounds

1. INTRODUCTION

In this paper, we present new approximation algorithms, lower bounds, and existence theorems for a class of network flows called *confluent flows*. A flow in a directed graph is said to be *confluent* if all the flow departing a node does so along a single outgoing edge.

Confluent flows arise in a number of scenarios including evacuation problems and various applications in networking. For instance, content delivery networks (CDNs) often organize their deployment of servers in the form of a rooted tree with each node forwarding data from its children to its parent and vice versa. Perhaps the most common application of confluent flows is in Internet routing. Most flows on the Internet today are confluent because Internet routing is primarily based on selecting a shortest path tree to each destination and then routing along the selected shortest paths;

*College of Computer & Information Science, Northeastern University, Boston, MA 02115. Email: {chenj,rraj}@ccs.neu.edu. Partially supported by NSF CAREER award CCR-9983901.

†MIT Department of Mathematics, Cambridge, MA 02139, Email: rdk@math.mit.edu. Supported by a Fannie and John Hertz Foundation Fellowship.

‡Microsoft Research, One Microsoft Way, Redmond, WA 98052. Email: lovasz@microsoft.com.

§College of Computer & Information Science, Northeastern University, Boston, MA 02115. Email: koods@ccs.neu.edu. Part of this work was done while the author was at Akamai Technologies, Cambridge, MA.

¶Department of Mathematics and School of Computer Science, McGill University, Montreal, Quebec, H3A 2A7, Canada. Email: vetta@math.mcgill.ca. Part of this work was done while the author was attending an algorithms workshop, supported by an ONR basic research grant, at Bell Labs.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

STOC'04, June 13–15, 2004, Chicago, Illinois, USA.

Copyright 2004 ACM 1-58113-852-0/04/0006 ...\$5.00.

thus, all packets departing a router for a particular destination depart along the same edge. A major shortcoming of shortest-paths routing, however, is that it ignores congestion at intermediate nodes and edges.

The main focus of this paper is on finding confluent flows with small congestion. Consider a directed graph G with k distinguished nodes, referred to as *sinks*, and non-zero demands on all the nodes of G . We would like to determine a confluent flow that routes every node demand to some sink such that the maximum flow arriving at any sink, referred to as the *congestion* of the sink, is minimized. If we drop the confluence constraint and thus allow *splittable flows*, then a flow that minimizes maximum congestion at a sink can be obtained in polynomial time by a straightforward reduction to the maximum flow problem. On the other hand, it was shown in [1] that minimizing confluent flow congestion is MAXSNP-hard, and that an $\tilde{O}(\sqrt{n})$ approximation is achievable for an n -node graph in the special case when all nodes have identical demands.

1.1 Our results

We present near-tight bounds on the approximability of confluent flows, and on the gap between confluent flows and splittable flows.

- The main result of this paper is a polynomial-time algorithm for determining a confluent flow with congestion at most $1 + \ln(k)$ in G , if G admits a splittable flow with congestion at most 1 (Section 4). We complement this result by presenting a graph G that admits a splittable flow with congestion at most 1, yet no confluent flow with congestion smaller than H_k (Section 3.1). Since $H_k = \ln k + \gamma - o(1)$, where γ is Euler's constant, we have resolved the gap between confluence and splittability to within an additive constant less than 1.

Our algorithm is based on a novel deterministic rounding of an optimal splittable flow, that repeatedly refines the flow by removing carefully selected edges and aggregating nodes into sinks, leading to the desired confluent flow. It is interesting to contrast the near-optimal bound achieved by our rounding scheme with the $\Omega(n^{1/4})$ bound achieved by a natural randomized rounding scheme, that selects for each node an outgoing edge with probability proportional to the flow on the edge in the splittable solution [1].

Since the optimal splittable flow congestion is a lower bound on the optimal confluent flow congestion, our algorithm achieves a $(1 + \ln k)$ -approximation for minimizing congestion. One may ask whether an improved approximation can be achieved efficiently.

- We show that it is NP-hard to approximate the congestion of an optimal confluent flow to within a factor of $(\lg k)/2$, thus resolving the polynomial-time approximability to within a multiplicative constant (Section 3.2)¹. It is interesting to note that our lower bound is not based upon a reduction from set cover [3] and relies on a weaker precondition than that used in the set cover hardness result.

While the bound of $1 + \ln(k)$ on the ratio between the congestion of confluent and splittable flows is existentially tight

¹Throughout this paper, we use $\lg$ to refer to $\log_2$.

up to an additive constant, it is natural to wonder if there are interesting classes of graphs for which the gap is smaller.

- A positive answer to this question is provided in Section 6, in which we prove that k -connected graphs with k sinks admit confluent flows of congestion less than $C + d_{\max}$, where C is the congestion of the best splittable flow. In particular, this means that the ratio between confluent and splittable congestion in k -connected graphs is at most 2. Interestingly, the proof of this existence theorem is non-constructive and relies on topological techniques introduced in [16].

Finally, we also consider a demand maximization problem, in which we seek a confluent flow with congestion at most 1 that maximizes the total demand of all the nodes whose demand is satisfied.

- We show that if G admits a splittable flow of congestion at most 1, then a variant of the congestion minimization algorithm yields a confluent flow in G with congestion at most 1 that satisfies $1/3$ fraction of total demand (Section 5). We also show that demand-maximization problem is NP-hard to approximate to a factor less than $4/3$ (Section 3.2).

Our 3-approximation result for demand maximization assumes the existence of a splittable flow that satisfies all demands with congestion at most 1. We can eliminate this assumption at the expense of increasing the approximation ratio by a constant factor by first determining a splittable flow of congestion at most 1 that satisfies a constant fraction of the maximum satisfiable demand; any of the algorithms of [2, 12] for single-source unsplittable flow would suffice for this purpose.

1.2 Related work

The bulk of our results in this paper compare confluent flows with a natural relaxation, namely splittable flows, which are well-characterized by the celebrated max-flow min-cut theorem of Ford and Fulkerson [4, 5]; there is a vast literature on efficient algorithms for obtaining the maximum flow. Another relaxation of confluent flow is unsplittable flow, which requires that the demand for every source be routed along a single path. Both the congestion minimization and demand maximization versions of unsplittable flow may be approximated to within a constant factor using the algorithms of [2, 12]. The relationship between the *edge congestion* of confluent and unsplittable flows is addressed in [15], in which an $\Omega(n)$ separation is established, where n is the number of nodes.

In the special case where G is an undirected graph and all vertices have unit demand, finding a confluent flow of congestion $\leq C$ is equivalent to partitioning G into k connected subgraphs of size $\leq C$, each containing one of the sinks. (Given such a partition, a confluent flow is obtained by routing all flow along the edges of a spanning tree in each subgraph.) When G is k -vertex-connected, Frank [7] conjectured in 1975 that such partitions always exist, provided that $kC \geq n$. In fact, he made the much stronger conjecture that given sinks $s_1, \dots, s_k$ and positive integers $n_1, \dots, n_k$ summing to n , one could partition G into k connected subgraphs, such that the i -th subgraph contains s_i and has *exactly* n_i vertices. This conjecture was proved independently by Lovász [16] and Győry [10]. Lovász's proof

applies also to directed graphs. Our result on the existence of confluent flows in k -connected graphs can be viewed as a weighted generalization of this theorem, in which the vertices are given non-negative real weights (demands) and one seeks to partition G into connected subgraphs whose total weights approximate a specified k -tuple of target weights.

In this paper, we have entirely focused on single commodity confluent flows. Multicommodity and fractional variants of confluent flows are studied in [1]. Multicommodity confluent flows are considered by [8, 13] in a model where the demands are not associated with individual source-sink pairs; instead with sources or sinks, as a whole. Also related is the work of [11], which raises the problem of finding a subtree of a given network that can route a given set of multicommodity flow pairs with minimum congestion. The impact of confluence on IP routing is studied in [15] and [17].

2. CONFLUENT FLOW PROBLEM DEFINITIONS

Let $G = (V, E)$ denote a directed graph and $d : V \rightarrow \mathbb{R}_+$ denote a function specifying the demand at each vertex. We denote the total demand, $\sum_{v \in V} d(v)$, by D and the maximum demand, $\max_{v \in V} d(v)$, by $d_{\max}$. Let $S = \{s_1, \dots, s_k\} \subseteq V$ denote a set of k sinks. Any flow $f : E \rightarrow \mathbb{R}_+$ routing the demands to the sinks satisfies the flow conservation equation

$$\sum_{e=(v,w) \in E} f(e) - \sum_{e=(u,v) \in E} f(e) = d(v)$$

at every vertex $v \in V - S$. For any node v , define the *inflow* of v , $\text{in}(v)$, to be the sum of the flows on the edges into v . The congestion of f at a node v is now defined as $d(v) + \text{in}(v)$.

Without loss of generality, we assume that each sink has only incoming edges. (Suppose s_i has outgoing edges, we can add a sink vertex s'_i and an edge (s_i, s'_i) to G and remove s_i from S .) We also assume without loss of generality that the maximum congestion of any node of G is 1; otherwise, all the demands and flows can be scaled by the maximum congestion to satisfy this property.

We say that a flow f is confluent if for every node u , there exists at most one edge (u, v) that has positive flow (i.e., $f(u, v) > 0$). Thus, a confluent flow yields a subgraph of G consisting of disjoint components $\{T_1, \dots, T_k\}$, such that each T_i is an arborescence directed towards the root s_i . In any arborescence T_i , the maximum node congestion occurs at the sink s_i and equals the total demand in T_i , given by $d(T_i)$. We refer to the maximum node congestion $\max_i d(T_i)$ as the congestion of the confluent flow.

In this paper, we consider two optimization problems concerning confluent flows. In the *congestion minimization* problem, we seek a confluent flow with minimum congestion among all confluent flows that satisfy all demands. In the *demand maximization* problem, we seek a confluent flow that satisfies maximum total demand among all confluent flows that have congestion at most 1.

3. LOWER BOUNDS

In this section, we present three lower bound results. We first present an instance where the congestion of the optimal confluent flow is at least H_k times that of the optimal splittable flow. We then show that it is NP-hard to approximate

the minimum congestion confluent flow to within a factor of $\frac{1}{2} \lg k$ and the demand maximization problem to a factor less than $4/3$.

3.1 Confluent to splittable: H_k gap

Figure 1 shows an instance with splittable congestion 1 but where the optimal confluent flow has congestion at least H_k . The congestion of any confluent flow is at least H_k since the node with demand 1 induces a flow of H_k into any sink that drains it. We leave it as an easy exercise for the reader to see that the flow that splits the outgoing flow at each node as shown in the figure achieves a congestion of 1.

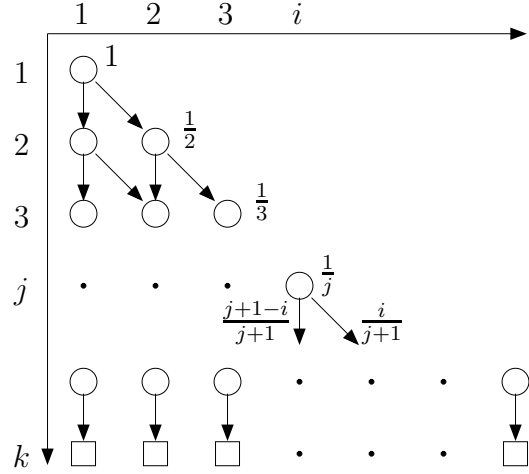


Figure 1: Instance demonstrating an H_k gap between confluent and splittable flows. The round nodes are sources with j nodes at level j each with supply $\frac{1}{j}$. The square nodes are sinks. The flows on the outgoing edges of the i th node on level j are specified.

3.2 Hardness of approximation

In [1] it was shown that the minimum congestion problem is NP-hard to approximate better than $\frac{4}{3}$ and hence MAXSNP-hard. Here, we refine the approach of [1] to improve the lower bound and show that it is NP-hard to approximate better than $(\lg k)/2$.

We present a hardness result for directed graphs with non-uniform demands. It is easy to modify this result to the case of uniform demands, where for each vertex v we wish to route exactly one unit of flow to a sink. Take a directed graph G with special vertices s_1, s_2, t_1, t_2 . It is known [6] that it is NP-hard to determine whether or not there are vertex-disjoint dipaths in G from s_1 to t_1 and from s_2 to t_2 . We show that any approximation algorithm for the confluent flow problem with performance guarantee better than $\frac{1}{2} \lg k$ can be used to determine in polynomial time whether or not such disjoint dipaths exist in G . This will give our result. We remark that the gadgets we use were first applied in [9] for the edge-disjoint path problem.

THEOREM 1. *It is NP-hard to approximate the optimal confluent flow congestion to a factor less than $\frac{1}{2} \lg k$.*

Proof. Given G we build an auxiliary network N as follows. Take a complete binary tree T on $2^{\lceil \lg k \rceil - 1} + 1$ nodes, with

root node r . We make T directed by replacing each edge with an arc directed away from the root. Then we replace each node v in the tree by a copy of G . We use the notation s_1^v , for example, to refer to the copy of s_1 in the copy of G associated to the node $v \in T$.

For a non-leaf node v in the tree, let $c_l(v)$ and $c_r(v)$ be its two children. Then the arc $(v, c_l(v))$ in T is replaced by the arc $(t_2^v, s_1^{c_l(v)})$ in the auxiliary network; similarly, the arc $(v, c_r(v))$ in T is replaced by the arc $(t_1^v, s_1^{c_r(v)})$ in the auxiliary network. For each leaf node u in the tree, we add the two arcs (t_1^u, t_1^{*u}) and (t_2^u, t_2^{*u}) , where t_1^{*u} and t_2^{*u} are sinks. Our construction is illustrated in Figure 2.

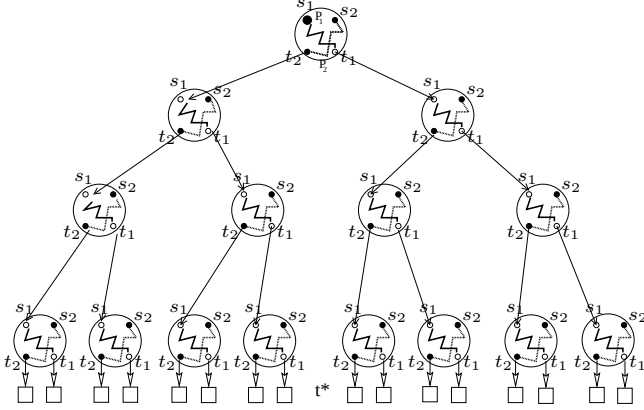


Figure 2: Instance used in the NP-hardness proof. The black vertices have demand one, except for the large black vertex s_1^r which has demand two, and the white vertices have demand zero.

In addition, we give each vertex in the auxiliary network a demand. Every copy of s_2 and t_2 has demand one. Every copy of s_1 and t_1 has demand zero, except for s_1^r which has demand two. Every other vertex has demand zero.

Now, suppose G contains vertex-disjoint dipaths P_1 and P_2 from s_1 to t_1 and from s_2 to t_2 , respectively. Utilizing these dipaths in each copy of G we obtain a collection of disjoint dipaths that end at the set of sinks t^* and that cover every copy of s_1, s_2, t_1 and t_2 . This is shown in Figure 2. Since each dipath contains only two vertices of non-zero demand (and exactly one for the dipath from s_1^r to t^*), the congestion of the resultant confluent flow is exactly two. This is optimal since there is a vertex with demand two.

Now suppose that G does not contain vertex-disjoint dipaths P_1 and P_2 from s_1 to t_1 and from s_2 to t_2 . Take any confluent flow in our network and consider the dipath P it induces from s_1^r to t^* . We now show that this dipath must have congestion at least $\lceil \lg k \rceil$. Towards this end, assume that P passes through copies of G corresponding to the nodes $r = v_1, v_2, \dots, v_{\lceil \lg k \rceil - 1}$ of T . Thus by construction P must pass through $s_1^{v_i}$, for $1 \leq i \leq \lceil \lg k \rceil - 1$. We claim that vertex $s_1^{v_i}$ has congestion at least $i + 1$. This is true for $s_1^{v_1}$. Assume then that $s_1^{v_j}$ has congestion at least $j + 1$, and consider the copy of G corresponding to v_j . We know that any flow at $s_1^{v_j}$ and at $s_2^{v_j}$ must be routed via $t_1^{v_j}$ or $t_2^{v_j}$. However, because there are no disjoint dipaths P_1 and P_2 , we must have one of the situations shown in Figure 3.2. Either the flow at $s_1^{v_j}$ is routed via $t_2^{v_j}$ and the flow at

$s_2^{v_j}$ is routed via $t_1^{v_j}$, or the flows at $s_1^{v_j}$ and at $s_2^{v_j}$ are both routed via the same vertex (either $t_1^{v_j}$ or $t_2^{v_j}$). In each case, the congestion at either $t_1^{v_j}$ or $t_2^{v_j}$ is at least $(j + 1) + 1$. This congestion is passed down to vertex $s_1^{v_{j+1}}$. Thus, by induction, it follows that the congestion of the vertex $s_1^{v_{\lceil \lg k \rceil}}$ is $\lceil \lg k \rceil$. Observe that the number of sinks in the auxiliary network is k .

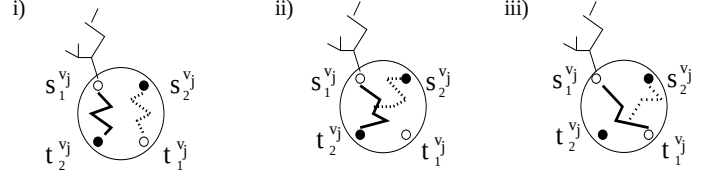


Figure 3: Routing in Networks without Disjoint Dipaths.

It follows that any approximation algorithm for the confluent flow problem with approximability guarantee better than $\frac{1}{2} \lg k$ can be used to determine in polynomial time whether a directed graph contains vertex disjoint dipaths from s_1 to t_1 and from s_2 to t_2 . $\square$

Finally, we show that the demand maximization problem is NP-hard to approximate better than $4/3$.

THEOREM 2. *It is NP-hard to approximate the confluent flow demand maximization problem to a factor less than $4/3$.*

PROOF. We'll use the same graph G as in the proof of Theorem 1, with demands of $1/2 - \varepsilon$ at s_1 and t_2 , $1/2 + \varepsilon$ at s_2 and t_1 , and 0 at every other node. If G contains vertex-disjoint dipaths from s_1 to t_1 and from s_2 to t_2 , we can route all the flow along these paths to achieve congestion 1. If not, then there is no way to achieve congestion 1 without excluding at least one of the sources of non-zero demand. Thus it is NP-hard to distinguish whether the maximum satisfiable demand is equal to 2 or to $3/2 + \varepsilon$. As $\varepsilon \rightarrow 0$, this gives the $4/3$ bound stated in the theorem. $\square$

4. CONGESTION MINIMIZATION

In this section, we present a polynomial-time algorithm to determine a confluent flow that satisfies all demands and has congestion at most $1 + \ln(k)$. We present our algorithm and its analysis in two stages. We first describe in Section 4.1 an algorithm that achieves a congestion of $1 + \lg k$. In Section 4.2, we refine the algorithm of Section 4.1 and its analysis to obtain the desired $1 + \ln k$ congestion bound.

4.1 A $(1 + \lg k)$ -congestion algorithm

We begin by giving a brief overview of the algorithm. Our starting point is a (splittable) flow f in G that routes all demands to the sinks and has maximum node congestion 1. The preceding flow can be found in polynomial time using a standard maximum flow algorithm. Without loss of generality, we may assume that the given directed graph G is the directed acyclic graph (dag) induced by the splittable flow.

As the algorithm proceeds, it transforms the graph G and the flow f by repeatedly performing one of three operations:

(i) aggregate a node into a sink if all of the outgoing edges of that node are to the same sink; (ii) remove an edge (often by breaking certain undirected cycles²) and redirect flow; and (iii) deactivate a sink by removing all edges incident into the sink and redirecting flow. These operations are illustrated in Figure 4. While these operations repeatedly make changes to the graph G (edges and nodes are removed), the set S of sinks (sinks are deactivated), the flow f , and the demands d (nodes are removed), we always maintain the following invariants:

1. f always satisfies d and the flow conservation constraints.
2. Congestion at any node $v \in V(G) - S$ never increases.
3. There are only incoming edges at each sink.

At termination, the transformed graph consists of k nodes, each node representing a set of nodes in the original graph that have been aggregated into a sink (including the sink). Any spanning forest of the edges contracted during the aggregation process yields the disjoint trees $\{T_1, \dots, T_k\}$ forming the desired confluent flow.

We are now ready to define our algorithm CONFLT. At any stage of CONFLT, for any sink s_i , let b_i denote the congestion of s_i (note that this is simply $d(s_i) + \text{in}(s_i)$). A node v is referred to as a *frontier node* if v has an edge incident into one of the sinks.

CONFLT(G, S, d, f): While $V(G) \neq S$, execute the following main loop:

1. **Node aggregation:** If a frontier node v has all of its outgoing edges going into one sink s_i , mark any one of these edges, contract v into s_i , and add $d(v)$ to $d(s_i)$. Repeat the main loop.
2. **Breaking sawtooth cycles:** Let $\hat{G}$ denote the graph obtained from G by adding an edge $e_{\text{rev}} = (s, v)$ for each edge $e = (v, s)$ from a frontier node to a sink. If $\hat{G}$ contains a directed simple cycle C of length > 2 , update f as follows. Let $f_{\min}$ denote the minimum flow value on any edge of $C \cap E(G)$. For each edge e of G , if $e \in C$ decrease $f(e)$ by $f_{\min}$ and if $e_{\text{rev}} \in C$ increase $f(e)$ by $f_{\min}$. Remove all edges with zero flow and repeat the main loop.
3. **Sink deactivation:** Let s_j be a sink node with only one adjacent frontier node v , and let s_ℓ be another sink node adjacent to v . If $b_j + f(v, s_\ell) < b_\ell - f(v, s_\ell)$, remove edge (v, s_ℓ) and send all its flow along edge (v, s_j) ; otherwise, remove edge (v, s_j) , send all its flow along edge (v, s_ℓ) , and deactivate s_j . Repeat the main loop.
4. **Output:** Output the marked edges.

Each node aggregation and sink deactivation step can be executed by a scan through the adjacency lists of the nodes. The step of breaking sawtooth cycles can be executed by constructing $\hat{G}$ and then performing a depth-first traversal of $\hat{G}$, both of which take time linear in the size of G . In each iteration, the size of the graph decreases, thus yielding a polynomial bound on the running time of CONFLT.

²We note that cycle-breaking is also an important component of the approximation algorithms for unsplittable flow [12, 2].

THEOREM 3. *Given a splittable flow with node congestion 1 on a graph with k sinks, CONFLT finds a confluent flow that satisfies all demands and has congestion at most $1 + \lg k$.*

Our proof is by a potential function argument. We define the potential of sink s_i as $\phi(s_i) = 2^{b_i}$, and the potential of the flow as the sum of the potentials of the active sinks. In the following sequence of lemmas, we show that the potential of the flow never increases.

LEMMA 4 (NODE AGGREGATION). *If a frontier node has all of its outgoing edges going into one sink, then the splittable flow can be modified so that a node can be removed from G without increasing node congestion.*

PROOF. Aggregating the frontier node v into a sink s_i increases $d(s_i)$ by $d(v)$ but does not change b_i . The node v is removed, and the flow along all the other edges remains unchanged. $\square$

LEMMA 5 (BREAKING SAWTOOTH CYCLES). *If there is a directed simple cycle C of length > 2 in $\hat{G}$, then the operation in step 2 does not increase the congestion of any node, and removes at least one edge.*

PROOF. At least one edge of $C \cap E(G)$ has $f_{\min}$ units of flow prior to performing the operation, and this edge is removed. To prove that the operation doesn't increase the congestion of any node, let v be any node of G . If v does not belong to C , then the flow on edges incident to v is unchanged. If v belongs to C , then C has two edges incident to v , and these are either two edges of G , or an edge of G and the reverse of another edge. In the first case, the congestion of v decreases by $f_{\min}$; in the second case, it is unchanged. $\square$

LEMMA 6 (SINK DEACTIVATION). *If none of the preconditions of Lemmas 4 and 5 hold, then the sink deactivation step can be performed, and it either removes an edge or deactivates a sink without increasing the potential of the resultant flow. Furthermore, the potential of the deactivated sink, if any, is no more than the potential of the flow before performing the sink deactivation.*

PROOF. If the preconditions of Lemmas 4 and 5 do not hold, then there is a sink node s_j which is adjacent to only one frontier node v . Indeed, we may find such a sink node by walking along edges of $\hat{G}$, always obeying the rule that never go to a sink if possible and that an edge (u, v) is never followed by (v, u) if possible. Every vertex of $\hat{G}$ has at least one outgoing edge, so the walk will go on indefinitely. Let v be the first vertex visited twice. By assumption $\hat{G}$ has no simple cycles of length > 2 , so the last two hops of the walk were (v, w) and (w, v) for some vertex w , and all outgoing edges of w lead to v . Either v or w must be a sink (else (v, w) and (w, v) are both in $E(G)$ contradicting the fact that G is a dag), and v cannot be a sink (as then w would satisfy the precondition of Lemma 4). Hence w is a sink s_j , and v is the only adjacent frontier node. Moreover, v is adjacent to at least one other sink s_ℓ ; else v would satisfy the precondition of Lemma 4.

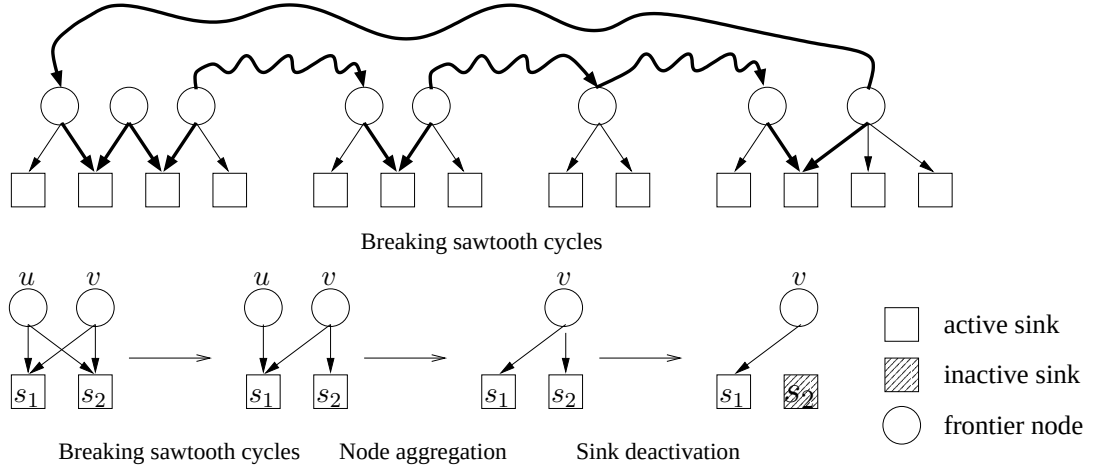


Figure 4: Illustrating steps 2, 1, and 3 of CONFLT. In the top figure, the bold edges form a sawtooth cycle.

If $b_j + f(v, s_\ell) < b_\ell - f(v, s_\ell)$, we remove (multi)edge (v, s_ℓ) and increase the flow on (v, s_j) by $f(v, s_\ell)$. It follows from the convexity of ϕ that the potential does not increase.

If $b_j + f(v, s_\ell) \geq b_\ell - f(v, s_\ell)$, we remove the (multi)edge (v, s_j) , increase the flow on (v, s_ℓ) by $f(v, s_j)$, and deactivate s_j . The increase in potential is at most

$$\begin{aligned} 2^{b_\ell + f(v, s_j)} - 2^{b_\ell} - 2^{b_j} &\leq 2^{b_\ell + f(v, s_j)} - 2^{b_\ell} - 2^{b_\ell - 2f(v, s_\ell)} \\ &\leq 2^{b_\ell + f(v, s_j)} - 2^{b_\ell - f(v, s_\ell) + 1} \leq 0. \end{aligned}$$

(The second step follows from the convexity of the potential function and the last step holds since $f(v, s_j) + f(v, s_\ell)$ is at most the congestion of v , which is at most 1.)

For the second part of the lemma, it suffices to note that the potential of the deactivated sink is less than the potential of the flow before the deactivation step. $\square$

Proof of Theorem 3: We first observe that for any non-sink node CONFLT marks exactly one outgoing edge (during a node aggregation step). Thus, the set of marked edges form a confluent flow. We now prove that the congestion of this flow is at most $\lg k + 1$. The potential of the flow at the start of the algorithm is at most $2k$. Lemmas 4 through 6 show that the potential of the flow is never increased. Furthermore, by Lemma 6, the potential of any deactivated sink is at most $2k$. Therefore, at termination, the potential of any sink is at most $2k$. The congestion of any sink in the final confluent flow is at most $1 + \lg k$. $\square$

4.2 An improved upper bound

In this section, we present a refinement IMP-CONFLT of the algorithm CONFLT and show that IMP-CONFLT achieves $1 + \ln k$ congestion. IMP-CONFLT differs from CONFLT in the sink deactivation step only. The remaining steps, namely breaking sawtooth cycles and node aggregation, are all identical to those in CONFLT.

We now present the sink deactivation step for IMP-CONFLT. As in the analysis of CONFLT, we maintain a potential for each sink. The potential of a sink with congestion x is given by $\phi(x) = e^x$. The potential of the system is defined to be the sum of the potentials of the active sinks.

3. **Parsimonious sink deactivation:** Find an induced subgraph $G_1 \subseteq G$ such that every edge in G_1 joins a

frontier node to a sink, G_1 contains all the outgoing edges of its frontier nodes and all the incoming edges of its sink nodes, and every frontier node in G_1 is adjacent to at least two sinks. Let S_1 be the set of sinks in G_1 .

- **Balancing:** Redistribute the flow from frontier nodes in G_1 to S_1 so that $\sum_{s_i \in S_1} \phi(b_i)$ is minimized. Remove any edges with zero flow.
- Find the sink $s \in S_1$ with minimum total in-flow. For any frontier node v adjacent to s , redirect the flow along any edge (v, s) to an arbitrary sink adjacent to v . Remove all edges adjacent to s , deactivate s , thus setting $S_1 \leftarrow S_1 - \{s\}$.
- Repeat **Balancing** and then repeat the main loop of IMP-CONFLT.

To complete the description of the parsimonious sink deactivation step, we need to specify (i) how the subgraph G_1 is determined and (ii) how the step **Balancing** is implemented. For a directed (multi)graph D , let the *undirected graph underlying D* denote the undirected graph with vertex set $V(D)$ and with an edge for each pair of vertices which are joined (in either order) by a directed edge of D . The construction of G_1 is given by the proof of the following lemma.

LEMMA 7. *If none of the preconditions of Lemmas 4 and 5 hold, then G contains a subgraph G_1 meeting the preconditions of the parsimonious sink deactivation step. Moreover, the undirected graph underlying G_1 is a tree.*

PROOF. Recall the graph $\hat{G}$ defined in step 2 of Algorithm CONFLT. Let H be the directed graph obtained from $\hat{G}$ by contracting all edges from frontier nodes to sinks and vice-versa, while retaining all other edges. (Therefore H may contain self-loops.) Each vertex of H corresponds to a set of vertices in $\hat{G}$, and the induced subgraph on this subset of $V(\hat{G})$ is strongly connected. Therefore, any simple cycle C in H may be expanded into a simple cycle $\hat{C}$ in $\hat{G}$ by replacing each vertex in C with a simple path in $\hat{G}$. Moreover, $\hat{C}$ has length > 2 , because if it contained only two edges, both of them would be contracted in forming H .

Therefore, if the precondition of Lemma 5 does not hold, H is acyclic. Let s be a node of H with no outgoing edges, let $\hat{G}_1$ be the induced subgraph of $\hat{G}$ corresponding to s , and let $G_1 = \hat{G}_1 \cap G$. Then G_1 is an induced subgraph of G , every edge of which joins a frontier node to a sink. If G_1 failed to contain one of the outgoing edges of its frontier nodes, then this edge e would be an outgoing edge from s in H , violating the construction. It is also clear that G_1 contains all the incoming edges of its sinks, because these edges are all contracted in forming H . Every frontier node in G_1 is adjacent to at least two sinks because otherwise the precondition of Lemma 4 would hold. Finally, T contains no cycles, because such cycles would correspond to cycles in $\hat{G}$ violating the precondition of Lemma 5. $\square$

For **Balancing**, we note that the minimization problem in the step is to minimize a convex cost function subject to certain linear constraints. In the full paper, we show that the unique minimum of $\sum_{s_i \in S_1} e^{b_i}$ is identical to that of any other strictly convex function, and can be obtained by a polynomial time algorithm based on max flow.

We have thus shown that the parsimonious sink deactivation step can be implemented in polynomial time, implying that IMP-CONFLT is a polynomial-time algorithm.

In the following, we prove that the net change in potential as a result of the parsimonious sink deactivation step is nonpositive. Let G_1 be as defined at the start of the parsimonious sink deactivation step and let G'_1 be the graph obtained after applying the balancing step to G_1 . Let K denote any weakly connected component of G'_1 . By Lemma 7, the undirected graph underlying G_1 is a tree. Since the balancing step can only remove edges from G_1 , the undirected graph underlying K is also a tree. It also follows from the analysis of the balancing step that every sink in K has the same congestion. Let b denote this congestion. For any edge (v, s) in K from a frontier node v to sink s , we define the *subtree rooted at (v, s)* , $T_{(v, s)}$, as the tree consisting of all nodes that can be reached from v via undirected paths passing through the edge (v, s) .

LEMMA 8. *Let v denote an arbitrary frontier node and let s denote an adjacent sink in K . Let β denote the smallest in-flow in K and $\alpha \leq \beta$ be a real number such that the flow on edge (v, s) is at most $1 - \alpha$. Then, we can inject an additional flow of α on edge (v, s) and assign the resultant additional demand to sinks in $T_{(v, s)}$ such that the additive increase in potential is at most $(\alpha/\delta)(\phi(b + \delta) - \phi(b))$, for any $\delta \in [1 - \beta, \alpha]$.*

PROOF. The proof is by induction on the number of sinks in K . The assumptions about K require that there be at least two sinks in K . So, for the induction base, we consider the case when K has exactly two sinks. Since the sum of the in-flows of the two sinks is at most 1, it follows that $\beta \leq 1/2$. Since $\alpha \leq \beta$, $[1 - \alpha, \beta]$ is nonempty exactly when $\alpha = \beta = 1/2$. We can inject an additional flow of $1/2$ into either of the sinks and increase the potential by at most $\phi(b + 1/2) - \phi(b)$, which completes the desired claim.

We now consider the induction step. If $[1 - \beta, \alpha]$ is empty, then there is nothing to prove. Otherwise, let δ be any real in $[1 - \beta, \alpha]$. We are given a frontier node v and an adjacent sink s such that the flow along (v, s) is at most $1 - \alpha$. Let v_1 through v_j denote j frontier nodes other than v that are adjacent to s . For $1 \leq \ell \leq j$, we select $\alpha_\ell \leq f(v_\ell, s)$ such that $\sum_\ell \alpha_\ell = \alpha - \delta$. Such a selection is well-defined since

the sum of the flows along these edges is at least $\beta - (1 - \alpha)$, which is at least $\alpha - \delta$ since $\delta \geq 1 - \beta$.

We increase flow along edge (v, s) by α and decrement the flow along the edges (v_ℓ, s) by α_ℓ . This increases the congestion of sink s by δ . The decrease in the flow along the edges (v_ℓ, s) implies that an additional flow of α_ℓ needs to be redirected to other sinks. For each frontier node v_ℓ , we select an arbitrary adjacent sink $s_\ell \neq s$ and inject an additional flow of α_ℓ along edge (v_ℓ, s_ℓ) into the subtree rooted at edge (v_ℓ, s_ℓ) (i.e., subtree $T_{(v_\ell, s_\ell)}$). Let β_ℓ denote the smallest in-flow in the subtree rooted at edge (v_ℓ, s_ℓ) . Since $\alpha_\ell \leq \alpha$ and $\alpha \leq \beta \leq \beta_\ell$, it follows that $\alpha_\ell \leq \beta_\ell$. By the induction hypothesis, we know that the total increase in potential of the sinks in this subtree is at most

$$\frac{\alpha_\ell}{\delta'} (\phi(b + \delta') - \phi(b)),$$

for any $\delta' \in [1 - \beta_\ell, \alpha_\ell]$. Since $\beta_\ell \geq \beta$ and $\delta \geq 1 - \beta$, we obtain that $\delta \geq 1 - \beta_\ell$. We consider two cases. If $\delta \leq \alpha_\ell$, it follows from the induction hypothesis that the total increase in potential in the subtree is at most

$$\frac{\alpha_\ell}{\delta} (\phi(b + \delta) - \phi(b)).$$

If $\delta > \alpha_\ell$, then we can simply increase the in-flow into sink s_ℓ by α_ℓ and achieve a potential increase of at most

$$(\phi(b + \alpha_\ell) - \phi(b)) \leq \frac{\alpha_\ell}{\delta} (\phi(b + \delta) - \phi(b)),$$

the last inequality following from Lemma 10 since $\delta > \alpha_\ell$.

By adding over all ℓ , we obtain that the total potential increase over all trees $T_{(v_\ell, s_\ell)}$ is at most

$$\frac{\sum_\ell \alpha_\ell}{\delta} (\phi(b + \delta) - \phi(b)).$$

On adding the potential increase of sink s , which is $\phi(b + \delta) - \phi(b)$, we obtain a total increase of at most

$$\frac{\alpha}{\delta} (\phi(b + \delta) - \phi(b)),$$

which completes the induction step. $\square$

LEMMA 9. *The parsimonious sink deactivation step does not increase the potential.*

PROOF. Let s^* denote a sink in K with minimum in-flow. Let α denote the in-flow of s^* and let v_1 through v_j denote the frontier nodes adjacent to s^* . Let α_ℓ denote the flow on edge (v_ℓ, s^*) , for $1 \leq \ell \leq j$. Let $s_\ell \neq s^*$ denote an arbitrary sink adjacent to v_ℓ . (Since every frontier node in K has at least two adjacent nodes, s_ℓ exists.) We redirect flow α_ℓ into the subtree $T_{(v_\ell, s_\ell)}$. Since the congestion of v_ℓ is at most 1, it follows that the flow along edge (v_ℓ, s_ℓ) is at most $1 - \alpha_\ell$. We consider two cases. If $\alpha_\ell < 1 - \alpha$, then we assign α_ℓ to sink s_ℓ , thus increasing its congestion to $b + \alpha_\ell$. Thus, the increase in potential of $T_{(v_\ell, s_\ell)}$ is at most

$$\phi(b + \alpha_\ell) - \phi(b) \leq \frac{\alpha_\ell}{1 - \alpha} (\phi(b + 1 - \alpha) - \phi(b)),$$

by Lemma 10.

We now consider the case $\alpha_\ell \geq 1 - \alpha$. Let β_ℓ denote the minimum in-flow of a sink node in $T_{(v_\ell, s_\ell)}$. Since $\beta_\ell \geq \alpha$, it follows that $1 - \alpha \geq 1 - \beta_\ell$. We invoke Lemma 8, with $\delta = (1 - \alpha)$ to obtain that the increase in potential of $T_{(v_\ell, s_\ell)}$ is at most

$$\frac{\alpha_\ell}{1 - \alpha} (\phi(b + 1 - \alpha) - \phi(b)).$$

Adding over all ℓ , and subtracting the potential of the deactivated sink s^* , we obtain the net change in the total potential of the system to be at most

$$\begin{aligned} & \frac{\alpha}{1-\alpha} (\phi(b+1-\alpha) - \phi(b)) - \phi(b) \\ &= e^b \left(\frac{\alpha}{1-\alpha} (e^{1-\alpha} - 1) - 1 \right) \\ &= e^b \left(\frac{\alpha}{1-\alpha} \left(\frac{1}{e^{\alpha-1}} - 1 \right) - 1 \right) \\ &\leq 0. \end{aligned}$$

(The last step uses the inequality $e^{\alpha-1} \geq 1 + \alpha - 1 = \alpha$.) $\square$

LEMMA 10. For $0 \leq x \leq 1$, the function $\frac{e^x-1}{x}$ is a monotonically increasing function of x .

PROOF. Let $g(x)$ equal $\frac{e^x-1}{x}$. Then $g'(x)$ is equal to

$$\frac{e^x}{x} - \frac{e^x}{x^2} + \frac{1}{x^2} = \frac{e^x(x-1)+1}{x^2} = \frac{(x-1)/e^{-x}+1}{x^2},$$

which is nonnegative since $e^{-x} \geq 1-x$ and $x \leq 1$. $\square$

THEOREM 11. Given a splittable flow with node congestion 1 on a graph with k sinks, there exists a polynomial time algorithm that finds a confluent flow satisfying the same demand and with node congestion not exceeding $1 + \ln k$.

PROOF. Lemmas 4, 5, and 9 show that the potential never increases. Since the initial total potential is ek , it follows that potential of any sink at termination is at most ek , implying a congestion of at most $1 + \ln k$. $\square$

5. DEMAND MAXIMIZATION

In the demand maximization problem, we need to find a routable subset of the nodes of maximum total demand, requiring a confluent flow. Suppose we are given a graph G that admits a splittable flow satisfying all demands, while incurring a congestion of at most 1. We present a polynomial time algorithm for finding a confluent flow in G of congestion at most 1 that satisfies demands of a subset of nodes whose demands add up to $1/3$ -factor of the total demand. We note that the preceding result assumes the existence of a splittable flow that satisfies all demands with congestion at most 1. As mentioned in Section 1, we can eliminate this assumption by first determining a splittable flow of congestion at most 1 that satisfies a constant fraction of the maximum satisfiable demand. For this purpose, we can use any of the constant-factor approximations known for demand maximization in single-source unsplittable flows [2, 12]; the best approximation ratio known is 4.43 [2]. This yields a 13.29-approximation for unconditional demand maximization in confluent flows.

The demand maximization algorithm MAXDEMAND is the same as CONFLT except the sink deactivation step and a postprocessing step.

3. **Sink deactivation:** Let s_j be a sink node with only one adjacent frontier node v , and let s_ℓ be another sink node adjacent to v .

- If $b_j - f(v, s_j) \leq \frac{1}{2}$, remove edge (v, s_ℓ) and send its flow along edge (v, s_j) .
- If $b_j - f(v, s_j) > \frac{1}{2}$, remove edge (v, s_j) , send its flow along edge (v, s_ℓ) and deactivate sink s_j .

After obtaining a set of disjoint arborescences $\{T_i\}_{i=1}^k$, the algorithm MAXDEMAND selects a subset of the nodes and satisfies their demands as follows. Let $\hat{b}_i$ denote the total demand in T_i (alternatively, the congestion of sink s_i) after the computation of the arborescences. For each T_i , if $\hat{b}_i \leq 1$, then satisfy all of the demand. If $1 < \hat{b}_i \leq \frac{3}{2}$, then greedily partition the nodes in T_i into groups whose total demands sum to at most $\frac{2}{3}\hat{b}_i$; select the nodes in the group with the maximum total demand and satisfy their demands using T_i . If $\hat{b}_i > \frac{3}{2}$, then search the nodes of T_i in any order and determine a subset of the nodes whose total demand is at least $1/2$ as follows.

- $\Delta_i \leftarrow 0$.
- While $\Delta_i < 1/2$ do
 - Visit the next node v .
 - If $d(v) \leq 1/2$, then mark v and $\Delta_i \leftarrow \Delta_i + d(v)$. Otherwise, unmark all marked nodes; mark v and $\Delta_i \leftarrow d(v)$.
- Select the marked nodes and satisfy their demands using T_i .

LEMMA 12. After the calculation of the arborescences, we have

$$\sum_{i: \hat{b}_i \leq 1} \hat{b}_i + \frac{1}{2} \sum_{i: 1 < \hat{b}_i \leq \frac{3}{2}} \hat{b}_i + \sum_{i: \hat{b}_i > \frac{3}{2}} \frac{1}{2} \geq \frac{D}{3}.$$

PROOF. We first show:

$$2 \sum_{i: \frac{1}{2} < \hat{b}_i \leq 1} \hat{b}_i + \frac{1}{2} \sum_{i: 1 < \hat{b}_i \leq \frac{3}{2}} \hat{b}_i \geq \sum_{i: \hat{b}_i > \frac{3}{2}} (\hat{b}_i - \frac{3}{2})$$

We plot the congestion of the sinks $\{b_i\}_{i=1}^k$ as a histogram. These values may change during the execution of the algorithm MAXDEMAND, specifically, in step 3. But the total area of the histogram is always D . Divide the xy -plane of the histogram into two sections by the $y = \frac{3}{2}$ line. Notice that when the algorithm redirects flow from one sink to another, demand moves from one sink to the other; in the histogram, this corresponds to a move of “area” either across the boundary or within a section. Denote by A the total area of the upper section. The area A cannot increase in step 3a since $b_j + f(v, s_\ell) \leq \frac{1}{2} + f(v, s_j) + f(v, s_\ell) \leq \frac{3}{2}$. Whenever A increases in step 3b, some sink s_j is deactivated and remains untouched thereafter, hence $\hat{b}_j = b_j - f(v, s_j)$. There are now three cases to consider.

- If $\frac{1}{2} < b_j - f(v, s_j) \leq 1$, then the increase in A is at most $1 \leq 2\hat{b}_j$.
- If $1 < b_j - f(v, s_j) \leq \frac{3}{2}$, then the increase in A is at most $f(v, s_j) - (b_j - \frac{3}{2}) < \frac{1}{2} < \frac{1}{2}\hat{b}_j$.
- If $b_j - f(v, s_j) > \frac{3}{2}$, then A does not increase.

Putting these together we obtain

$$2 \sum_{i: \frac{1}{2} < \hat{b}_i \leq 1} \hat{b}_i + \frac{1}{2} \sum_{i: 1 < \hat{b}_i \leq \frac{3}{2}} \hat{b}_i \geq \sum_{i: \hat{b}_i > \frac{3}{2}} (\hat{b}_i - \frac{3}{2})$$

Note that

$$\sum_{i: \frac{1}{2} < \hat{b}_i \leq 1} \hat{b}_i + \sum_{i: 1 < \hat{b}_i \leq \frac{3}{2}} \hat{b}_i + \sum_{i: \hat{b}_i > \frac{3}{2}} \hat{b}_i = D$$

The result immediately follows. $\square$

THEOREM 13. *Given a splittable flow with node congestion 1 on a graph with total demand D , MAXDEMAND finds a confluent flow with node congestion at most 1, satisfying demands of a subset of nodes, with total demand at least $\frac{D}{3}$.*

PROOF. Since the demands are routed with $\{T_i\}_{i=1}^k$, the flow is confluent. Since for each T_i , the algorithm selects total demand at most 1, the node congestion is at most 1. For T_i with $\hat{b}_i \leq 1$, all $\hat{b}_i$ demand is satisfied. For T_i with $1 < \hat{b}_i \leq \frac{3}{2}$, suppose the total demands of the groups are $d_1 \leq d_2 \leq \dots \leq d_r$. Only d_1 can be less than or equal to $\hat{b}_i/3$, otherwise d_1 and d_2 could have been combined. Since $d_1 + d_r > \frac{2}{3}\hat{b}_i$, it follows that $r = 2$ (otherwise $d_1 + d_2 + d_r > \hat{b}_i$). Therefore, $d_r \geq \hat{b}_i/2$, i.e. at least $\hat{b}_i/2$ is satisfied; totally $\frac{1}{2} \sum_{i: 1 < \hat{b}_i \leq \frac{3}{2}} \hat{b}_i$ is satisfied. For T_i with $\hat{b}_i > \frac{3}{2}$, at least $\Delta_i \geq 1/2$ demand is satisfied; totally at least $\sum_{i: \hat{b}_i > \frac{3}{2}} 1/2$ is satisfied. From Lemma 12, the algorithm satisfies at least $D/3$ demand in total. $\square$

6. CONFLUENT FLOWS IN K -CONNECTED GRAPHS

It turns out that if G is k -connected, then any splittable flow of congestion C may be replaced by a confluent flow of congestion $< C + d_{\max}$. In fact we will prove the following stronger theorem. Let us say that a directed graph G is k -connected to a set $S \subseteq V(G)$, if each vertex $v \notin S$ can be joined to S by k paths which are disjoint except for the common vertex v .

THEOREM 14. *Let G be a directed graph with sinks $S = \{s_1, \dots, s_k\}$, demands $d : V \rightarrow \mathbb{R}_+$, maximum demand $d_{\max}$, and total demand D . Suppose that G is k -connected to S . Given target values $t_1, \dots, t_k$ summing to D , there is a confluent flow in G such that $C(s_i) < t_i + d_{\max}$ for all i .*

In particular, taking $t_1 = \dots = t_k = D/k < C$, we obtain a confluent flow of congestion $< C + d_{\max}$ as claimed. Theorem 14 can be seen as a weighted version of the following theorem.

THEOREM 15 (LOVÁSZ, 1977 [16]). *Let G be a digraph, $v_1, \dots, v_k \in V(G)$ and assume that G is k -connected to S . Let, furthermore, k positive integers $n_1, \dots, n_k$ be given whose sum is $|V(G)|$. Then G contains k vertex-disjoint arborescences $A_1, \dots, A_k$, such A_i is rooted at v_i and $|V(A_i)| = n_i$.*

It is worthwhile to introduce here *nearly confluent* flows: these are splittable flows where the flow may split only at nodes with no incoming flow. Theorem 14 is a consequence of the following two facts about nearly confluent flows. First, if we allow nearly confluent flows, then the congestions at the sinks can be prescribed arbitrarily:

THEOREM 16. *Let G be a directed graph with sinks $S = \{s_1, \dots, s_k\}$, demands $d : V \rightarrow \mathbb{R}_+$ and total demand D . Given any non-negative target values $t_1, \dots, t_k$ summing to D , there is a nearly confluent flow f in G such that $C(s_i) = t_i$ for all i .*

Second, a nearly confluent flow may be rounded to a confluent flow without increasing the maximum edge congestion by too much.

THEOREM 17. *If f is a nearly confluent flow in G , then f may be rounded to a confluent flow $\tilde{f}$ such that $C_{\tilde{f}}(s_i) < C_f(s_i) + d_{\max}$ for all i .*

We'll obtain Theorems 14 and 16 using the topological techniques which appear in [16], where Theorem 15 is proved. The proof of Theorem 17 uses a simple rounding technique in the spirit of [14], and will appear in the full version of this paper.

The arborescence complex of a directed graph. In [16] a topological space $\mathcal{K}$ is associated with each directed graph G with distinguished vertex $\hat{a}$, known as the *arborescence complex* of G relative to $\hat{a}$. It can be modeled as a cellular complex whose vertices are in one-to-one correspondence with the arborescences of G rooted at $\hat{a}$. In this section we present a definition of the arborescence complex which is equivalent to the original, but makes clearer the relation with confluent flows.

DEFINITION 18. *Let $G = (V, E)$ be a directed acyclic graph with distinguished vertex $\hat{a}$. Assume every $v \in V(G)$ has a directed path to $\hat{a}$. Represent each $\hat{a}$ -rooted arborescence by a function $F : E(G) \rightarrow \mathbb{R}$ where $F(e) = 1$ if e belongs to the arborescence, 0 otherwise. A fractional arborescence in G is a function $F : E(G) \rightarrow [0, 1]$ which is a convex combination of arborescences.*

A fractional arborescence is called a *near-arborescence* if it satisfies the following property: if v is a vertex such that $F(e) > 0$ for at least two distinct outgoing edges e , then $F(e) = 0$ for all incoming edges e . In other words, a near-arborescence is a convex combination of arborescences, any of which can be transformed into any other by disconnecting and reattaching some leaves. The set of all near-arborescences in G is a subspace $\mathcal{K} \subseteq \mathbb{R}^{E(G)}$ called the *arborescence complex* of G relative to $\hat{a}$.

In Figure 6 we have illustrated the arborescence complex for a six-vertex graph G . In this case, a fractional arborescence is determined by specifying, for each of the three topmost vertices of G , a convex combination of the two outgoing edges. Thus the space of fractional arborescences in G is a cube: a product of three copies of Δ^1 , one for each of the three topmost vertices. The near-arborescences are those in which the topmost vertex does not feed any weight into an undecided vertex. The arborescence complex $\mathcal{K}$ is the subset of the cube where this criterion holds; it is an octagon consisting of eight edges of the cube.

The fundamental fact about the topology of arborescence complexes is the following theorem from [16].

THEOREM 19 ([16], THEOREM 4). *Let G be a digraph, $\hat{a} \in V(G)$, and assume that G is k -connected to $\hat{a}$ ($k \geq 2$). Then the arborescence complex $\mathcal{K}$ of G relative to $\hat{a}$ satisfies $\tilde{H}_0(\mathcal{K}) = \dots = \tilde{H}_{k-2}(\mathcal{K}) = 0$, where $\tilde{H}$ denotes reduced homology with integer coefficients.*

Here, as in [16], the importance of Theorem 19 is that it enables us to apply a generalized intermediate value theorem to obtain a near-arborescence with desired properties.

Proof of Theorem 16: Given a graph G with demands $d : V(G) \rightarrow \mathbb{R}_+$ and sinks $s_1, \dots, s_k$, extend it to a graph $\hat{G}$ by adjoining an auxiliary vertex $\hat{a}$ with incoming edges $e_1, \dots, e_k$ from $s_1, \dots, s_k$. In terms of $\hat{G}$, we want to show

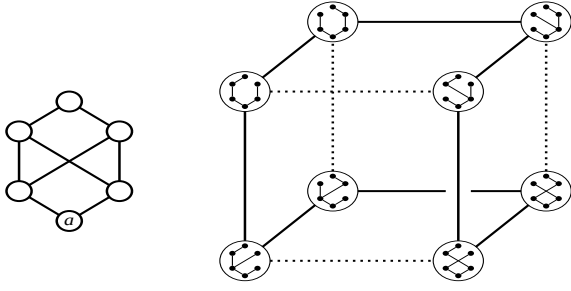


Figure 5: A graph and its arborescence complex

that there is a nearly confluent flow f in $\hat{G}$ satisfying $f(e_i) = t_i$ for $i = 1, \dots, k$.

If one is given demands $d : V(\hat{G}) \rightarrow \mathbb{R}_+$ and a near-arborescence F , this data naturally defines a flow $f : E(\hat{G}) \rightarrow \mathbb{R}_+$, routing all the demands to the sink vertex $\hat{a}$, according to the prescription that each vertex v distributes its outgoing flow in the proportions specified by F . (Note that the set of edges e with $F(e) > 0$ must constitute a DAG in order for this flow to be well-defined. If F is a near-arborescence, it is automatic that this edge set is a DAG.) For a fixed set of demands, the mapping which associates to each near-arborescence F the corresponding flow f constitutes a continuous function from $\mathcal{K}$ to the space of all flows in G . If F is an arborescence, the corresponding flow f is a confluent flow. If F is a near-arborescence, the corresponding flow is nearly confluent.

Let $\Delta \subseteq \mathbb{R}^k$ denote the $(k-1)$ -simplex

$$\Delta = \left\{ (x_1, \dots, x_k) \in \mathbb{R}_+^k : \sum_{i=1}^k x_i = D \right\}.$$

If $\mathcal{K}$ is the arborescence complex of $\hat{G}$ relative to $\hat{a}$, we may map $\mathcal{K}$ to Δ by mapping a near-arborescence F to the vector $(f(e_1), f(e_2), \dots, f(e_k))$, where f is the flow corresponding to F . This is a continuous function $\phi : \mathcal{K} \rightarrow \Delta$, and we will be done if we can find a point $F \in \mathcal{K}$ which maps to $(t_1, \dots, t_k)$. The existence of such a point is proved using the following generalized intermediate value theorem, whose proof uses standard techniques from homology theory and will appear in the full version of this paper.

THEOREM 20. *Let X be a finite-dimensional cellular complex. Suppose $\phi : X \rightarrow \Delta^r$ is a continuous map to an r -dimensional simplex, such that for every s -dimensional face $\sigma^s \subseteq \Delta^r$, the subspace $Y = \phi^{-1}(\sigma^s) \subseteq X$ is a non-empty subcomplex satisfying $\tilde{H}_0(Y) = \dots = \tilde{H}_{s-1}(Y) = 0$. Then ϕ is a surjection, i.e. every point of Δ^r is the image of some point in X under ϕ .*

To apply Theorem 20 to the arborescence complex $\mathcal{K}$ and the mapping $\phi : \mathcal{K} \rightarrow \Delta^{k-1}$ defined by $F \mapsto (f(e_1), \dots, f(e_k))$, we must verify that $\phi^{-1}(\sigma^s)$ satisfies the homological vanishing criterion specified in the theorem, for each face $\sigma^s \subseteq \Delta^{k-1}$. Each such face is determined by specifying $s+1$ of the incoming edges at $\hat{a}$ — without loss of generality, say $e_1, \dots, e_{s+1}$ — and requiring the flow to be zero on e_i for all $i > s+1$. Thus $\phi^{-1}(\sigma^s)$ is the arborescence complex of $G \cup \{e_1, \dots, e_{s+1}\}$ relative to $\hat{a}$. In this graph, no $v \neq \hat{a}$ can be separated from $\hat{a}$ by removing fewer than $s+1$ vertices, so Theorem 19 ensures the vanishing of the required homology groups. $\square$

7. REFERENCES

- [1] J. Chen, R. Rajaraman, and R. Sundaram. Meet and merge: Approximation algorithms for confluent flows. In *Proceedings of the 35th Annual ACM Symposium on Theory of Computing*, pages 373–382, June 2003.
- [2] Y. Dinitz, N. Garg, and M. Goemans. On the single-source unsplittable flow problem. In *Proceedings of the 39th Annual IEEE Symposium on Foundations of Computer Science*, pages 290–299, November 1998.
- [3] U. Feige. A threshold of $\ln n$ for approximating set cover. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 314–318, May 1996.
- [4] L. Ford, Jr. and D. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956.
- [5] L. Ford, Jr. and D. Fulkerson. *Flows in Networks*. Princeton University Press, Princeton, NJ, 1962.
- [6] S. Fortune, J. Hopcroft, and J. Wyllie. The directed subgraph homeomorphism problem. *Theoretical Computer Science*, 10:111–121, 1980.
- [7] A. Frank. *Combinatorial algorithms, algorithmic proofs*. PhD thesis, Eötvös University, Budapest, 1976. In Hungarian.
- [8] A. Gupta, J. Kleinberg, A. Kumar, R. Rastogi, and B. Yener. Provisioning a virtual private network: A network design problem for multicommodity flow. In *Proceedings of the 33rd Annual ACM Symposium on Theory of Computing*, 2001.
- [9] V. Guruswami, S. Khanna, R. Rajaraman, B. Shepherd, and M. Yannakakis. Near-optimal hardness results and approximation algorithms for edge-disjoint paths and related problems. *Journal of Computer and System Sciences*, 67:473–496, 2003.
- [10] E. Györi. On division of graphs to connected subgraphs. In *Proceedings of the Fifth Hungarian Combinatorial Colloquium*, Budapest, 1976.
- [11] S. Khuller, B. Raghavachari, and N. Young. Designing multi-commodity flow trees. *Information Processing Letters*, 50:49–55, 1994.
- [12] J. M. Kleinberg. Single-source unsplittable flow. In *Proceedings of the 37th Annual IEEE Symposium on Foundations of Computer Science*, pages 68–77, Oct. 1996.
- [13] A. Kumar, R. Rastogi, A. Silberschatz, and B. Yener. Algorithms for provisioning virtual private networks in the hose model. In *Proceedings of the ACM SIGCOMM Conference*, pages 135–148, 2001.
- [14] J. K. Lenstra, D. B. Shmoys, and E. Tardos. Approximation algorithms for scheduling unrelated parallel machines. *Math. Programming*, 46(3):259–271, 1990.
- [15] D. Lorenz, A. Orda, D. Raz, and Y. Shavitt. How good can IP routing be? Technical Report 2001-17, DIMACS, Apr. 2001.
- [16] L. Lovász. A homology theory for spanning trees of a graph. *Acta Mathematica Academiae Scientiarum Hungaricae*, 30(3-4):241–251, 1977.
- [17] J. Wang and K. Nahrstedt. Hop-by-hop routing algorithms for premium-class traffic in diffserv networks. In *Proceedings of IEEE INFOCOM*, New York, NY, June 2002.